

Aplicações Web

Professor Lozano

Tópicos que iremos abordar na aula:

- Requisitos de Software.
- Modelagem do Sistema (UML) / Arquitetura.

Aplicações Web

Professor Lozano

Passo 1 – Aplicação que será desenvolvida

O ImagemPeças é, na prática, um pequeno álbum de fotos de peças online. Qualquer pessoa pode criar uma conta informando nome, e-mail e senha, e depois entrar no sistema usando esse e-mail e senha. Uma vez logada, a pessoa passa a ter acesso a um acervo de imagens que pode consultar, adicionar e organizar.

Para guardar uma nova imagem de peça industrial, o usuário escolhe o arquivo no computador (uma foto nos formatos PNG, JPEG ou GIF), dá um nome a ela e a marca com algumas etiquetas (tags) que descrevem o conteúdo, por exemplo "placa mãe", "memória" ou "hd". O sistema mostra uma prévia da imagem antes de confirmar o envio, e depois de salva ela passa a fazer parte da galeria, disponível para consulta.

A galeria funciona como uma vitrine: as imagens das peças aparecem em miniaturas, com nome, tamanho e data de envio, e o usuário pode filtrar o que quer ver digitando uma palavra-chave (que busca tanto no nome quanto nas tags) ou escolhendo um formato específico de arquivo. Ao clicar em qualquer miniatura, a imagem abre em tamanho real em uma nova aba, podendo ser baixada. Quando termina de usar o sistema, o usuário pode sair da conta a qualquer momento pelo botão "Sair" no topo da página.

Área da Aplicação	Tecnologia Utilizada
Front-End	Next.js 13 / React 18 / TypeScript, Tailwind CSS, Formik + Yup
Back-End	Spring Boot 4 (Java 25), Spring Security, Spring Data JPA, JWT
Banco de Dados	PostgreSQL
Autenticação	Token JWT (HS256), stateless, expiração de 60 minutos
Empacotamento	Docker / docker-compose (4 serviços: app, sbootapp, db, pgadmin)

Aplicações Web

Professor Lozano

Passo 2 – Levantamento de Requisitos

Requisitos Funcionais

ID	Descrição
RF01	O sistema deve permitir que um visitante crie uma conta informando nome, e-mail e senha.
RF02	O sistema deve impedir o cadastro de um e-mail já existente.
RF03	O sistema deve permitir que um usuário cadastrado se autentique com e-mail e senha.
RF04	O sistema deve emitir um token de acesso (JWT) após autenticação bem-sucedida.
RF05	O sistema deve permitir que o usuário autenticado encerre sua sessão.
RF06	O sistema deve permitir que o usuário autenticado publique uma nova imagem, informando nome, tags e arquivo.
RF07	O sistema deve validar o formato (PNG, JPEG, GIF) e o tamanho do arquivo antes do envio.
RF08	O sistema deve permitir que o usuário pesquise imagens por nome, por tag e/ou por extensão.
RF09	O sistema deve exibir as imagens encontradas em formato de galeria, com nome, extensão, tamanho e data de upload.
RF10	O sistema deve permitir a visualização da imagem em tamanho real a partir da miniatura.
RF11	O sistema deve notificar o usuário sobre sucesso ou falha de cada operação realizada.

Requisitos Não-Funcionais

ID	Categoria	Descrição
RNF01	Segurança	Senhas devem ser armazenadas com hash (BCrypt); nunca em texto plano.
RNF02	Segurança	Operações de escrita (publicar imagem) exigem token JWT válido no cabeçalho Authorization.
RNF03	Confiabilidade	A sessão do usuário deve expirar automaticamente 60 minutos após a emissão do token
RNF04	Usabilidade	O sistema deve fornecer feedback visual (indicador de carregamento e notificações) durante operações assíncronas.
RNF05	Portabilidade	O sistema deve ser executável em qualquer ambiente com suporte a Docker, com front-end, back-end e banco isolados em containers.
RNF06	Interoperabilidade	O back-end deve expor uma API REST stateless, consumida via JSON e multipart/form-data.
RNF07	Manutenibilidade	O back-end deve manter separação em camadas (domínio, aplicação, infraestrutura), permitindo evolução incremental do sistema ao longo do semestre.

Aplicações Web

Professor Lozano

Passo 3 – Regras de Negócio

ID	Descrição
RN01	Não é permitido cadastrar dois usuários com o mesmo e-mail.
RN02	A senha do usuário deve ter no mínimo 8 caracteres.
RN03	O arquivo de imagem deve ter no máximo 4 MB.
RN04	Somente os formatos PNG, JPEG e GIF são aceitos.
RN05	Nome e tags da imagem são obrigatórios, com limite de 50 caracteres cada.
RN06	O token de acesso expira 60 minutos após ser emitido.

Passo 4 – Rastreabilidade: Requisito p/ Caso de Uso

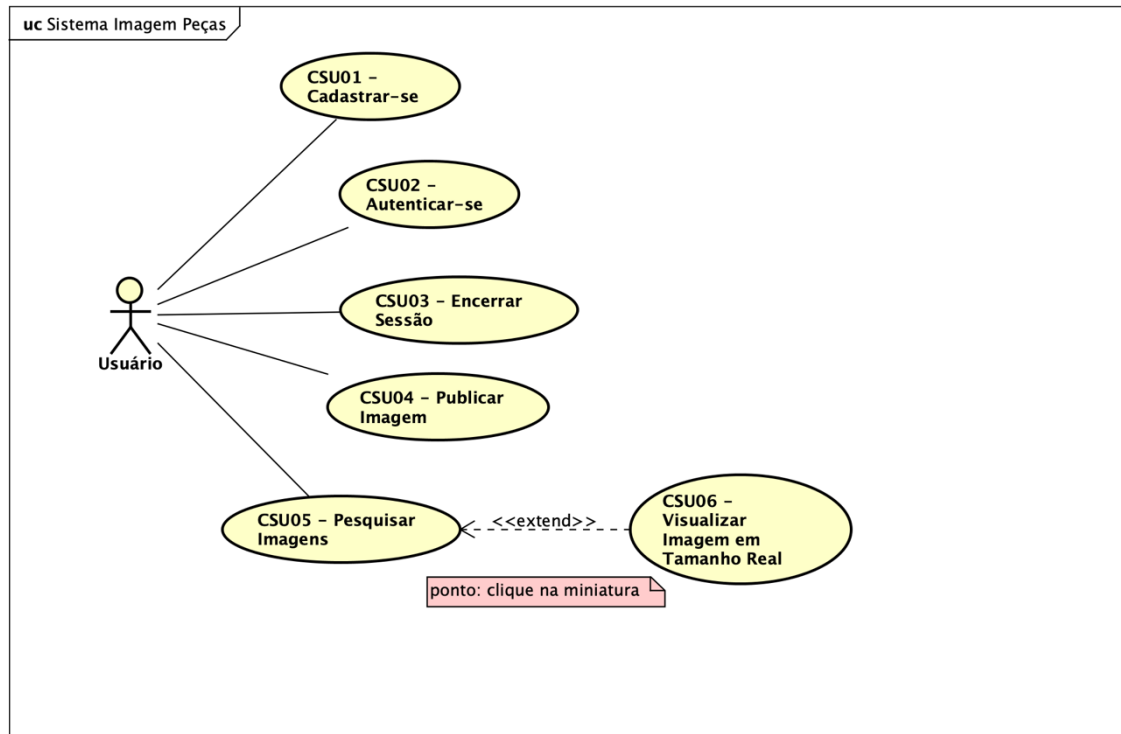
Requisito	Caso(s) de Uso
RF01, RF02	CSU01 – Cadastrar-se
RF03, RF04	CSU02 – Autenticar-se
RF05	CSU03 – Encerrar Sessão
RF06, RF07	CSU04 – Publicar Imagem
RF08, RF09	CSU05 – Pesquisar Imagens
RF10	CSU06 – Visualizar Imagem em Tamanho Real
RF11	Transversal – CSU01, CSU02, CSU04, CSU05

Aplicações Web

Professor Lozano

Passo 5 – Casos de Uso

Diagrama de Casos de Uso



Observações: apenas um ator de negócio existe no sistema (Usuário). CSU01 e CSU02 podem ser exercidos por um usuário ainda não autenticado; CSU03 a CSU06 exigem sessão ativa, essa condição está registrada como pré-condição em cada descrição de caso de uso que será visto a seguir, e não como um relacionamento «include» de “login” em todos os demais casos, para evitar o uso indevido de inclusão só para expressar pré-condição.

Aplicações Web

Professor Lozano

Passo 6 – Especificação dos Casos de Uso

CSU01 – Cadastrar-se

Campo	Descrição
Importância	Alta - pré-requisito para todos os demais casos de uso
Sumário	Permite que um visitante crie uma conta no sistema informando nome, e-mail e senha
Ator primário	Visitante
Pré-condições	O e-mail informado não pode estar previamente cadastrado
Fluxo principal	1. O visitante acessa a tela de login e seleciona “Sign Up”. 2. O sistema exibe o formulário de cadastro (nome, e-mail, senha, confirmação de senha). 3. O visitante preenche os dados e confirma. 4. O sistema valida os dados informados. 5. O sistema criptografa a senha e grava o novo usuário. 6. O sistema exibe notificação de sucesso e retorna à tela de login.
Fluxos alternativos	A1 (passo 3): o visitante seleciona “Cancel” → o sistema retorna à tela de login sem gravar dados.
Fluxos de exceção	E1 (passo 4): e-mail mal formatado, senha com menos de 8 caracteres, ou senhas não coincidentes → o sistema exibe o erro no campo correspondente e permanece no formulário. E2 (passo 5): e-mail já cadastrado (RN01) → o sistema responde HTTP 409 e exibe “User already exists!”; o cadastro não é realizado.
Pós-condições	um novo usuário foi persistido com senha criptografada (BCrypt).
Regras de negócio	RN01, RN02
Notas de implementação	<pre>POST /v1/users — UserController.save() → UserServiceImpl.save() → PasswordEncoder.encode() → UserRepository.save().</pre>

Aplicações Web

Professor Lozano

CSU02 – Autenticar-se

Campo	Descrição
Importância	Alta
Sumário	Permite que um usuário cadastrado obtenha acesso autenticado, informando e-mail e senha.
Ator primário	Usuário
Pré-condições	O usuário deve possuir cadastro prévio (CSU01)
Fluxo principal	1. O usuário informa e-mail e senha na tela de login. 2. O usuário confirma o login. 3. O sistema valida as credenciais. 4. O sistema gera um token de acesso válido por 60 minutos (RN06). 5. O sistema armazena a sessão no navegador do usuário. 6. O sistema redireciona o usuário para a galeria de imagens (CSU05).
Fluxos alternativos	Não se aplica.
Fluxos de exceção	E1 (passo 3): e-mail não cadastrado ou senha incorreta → HTTP 401, notificação “User or password are incorrect!”; usuário permanece na tela de login.
Pós-condições	O usuário possui um token JWT válido armazenado localmente e é redirecionado à galeria.
Regras de negócio	RN06
Notas de implementação	POST /v1/users/auth – UserController.authenticate() → UserServiceImpl.authenticate() (BCrypt.matches + JwtService.generateToken()); front-end grava a sessão em localStorage["_auth"].

CSU03 – Encerrar Sessão

Campo	Descrição
Importância	Baixa
Sumário	Permite que o usuário autenticado encerre sua sessão
Ator primário	Usuário autenticado
Pré-condições	Usuário autenticado (CSU02 concluído)

Aplicações Web

Professor Lozano

Fluxo principal	1. O usuário seleciona “Sair” no cabeçalho. 2. O sistema remove a sessão local do usuário. 3. O sistema redireciona o usuário para a tela de login.
Fluxos alternativos	Não se aplica.
Fluxos de exceção	Não se aplica.
Pós-condições	A sessão local foi invalidada; o token permanece tecnicamente válido no servidor até expirar naturalmente.
Regras de negócio	Não se aplica.
Notas de implementação	operação inteiramente client-side (Header.logout() → AuthService.invalidateSession()); não há chamada ao back-end neste caso de uso.

CSU04 – Publicar Imagem

Campo	Descrição
Importância	Alta - funcionalidade central do sistema
Sumário	Permite que o usuário autenticado envie uma nova imagem ao acervo, informando nome, tags e arquivo.
Ator primário	Usuário autenticado
Pré-condições	Usuário autenticado (token JWT válido)
Fluxo principal	1. O usuário seleciona “Add New” na galeria. 2. O sistema exibe o formulário de nova imagem. 3. O usuário informa nome, tags e seleciona o arquivo. 4. O sistema exibe a pré-visualização da imagem. 5. O usuário confirma o envio (“Save”). 6. O sistema valida os dados informados. 7. O sistema envia o arquivo e os metadados ao servidor, autenticado por token. 8. O sistema grava a imagem no banco de dados. 9. O sistema exibe notificação de sucesso e limpa o formulário.
Fluxos alternativos	A1 (passo 3): usuário seleciona “Cancel” → retorna à galeria sem enviar dados.

Aplicações Web

Professor Lozano

Fluxos de exceção	E1 (passo 6): nome/tags vazios ou acima de 50 caracteres, arquivo ausente, maior que 4 MB, ou fora dos formatos aceitos → erro exibido no campo, envio bloqueado. E2 (passo 7): token ausente ou expirado → HTTP 401/403; usuário deve autenticar-se novamente (retorno a CSU02).
Pós-condições	Uma nova imagem foi persistida (bytes + metadados) e passa a ser recuperável via pesquisa (CSU05).
Regras de negócio	RN03, RN04, RN05
Notas de implementação	POST /v1/images (multipart/form-data, cabeçalho Authorization: Bearer <token>) — ImagesController.save() → ImageMapper.mapToImage() → ImageServiceImpl.save().

CSU05 – Pesquisar Imagens

Campo	Descrição
Importância	Alta
Sumário	Permite pesquisar e visualizar, em formato de galeria, as imagens cadastradas, filtrando por nome/tag e/ou extensão.
Ator primário	Usuário autenticado.
Pré-condições	Usuário autenticado.
Fluxo principal	1. O sistema exibe a tela de galeria. 2. O usuário informa um termo de busca e/ou seleciona uma extensão. 3. O usuário confirma a pesquisa (“Search”). 4. O sistema consulta as imagens que atendem aos filtros. 5. O sistema exibe as imagens em miniaturas, com nome, extensão, tamanho e data de upload.
Fluxos alternativos	A1 (passo 2): nenhum filtro informado → o sistema retorna todas as imagens cadastradas.
Fluxos de exceção	E1 (passo 4): nenhuma imagem atende aos filtros → notificação “No results found!”; galeria permanece vazia.
Pós-condições	A galeria exibe o resultado da última pesquisa.
Regras de negócio	Não se aplica.
Ponto de extensão	“Clique na miniatura” → CSU06.

Aplicações Web

Professor Lozano

Notas de implementação	GET /v1/images?query=&extension= — ImagesController.search() → ImageRepository (Specifications nameLike / tagsLike / extensionEqual).
-------------------------------	--

CSU06 – Visualizar Imagem em Tamanho Real

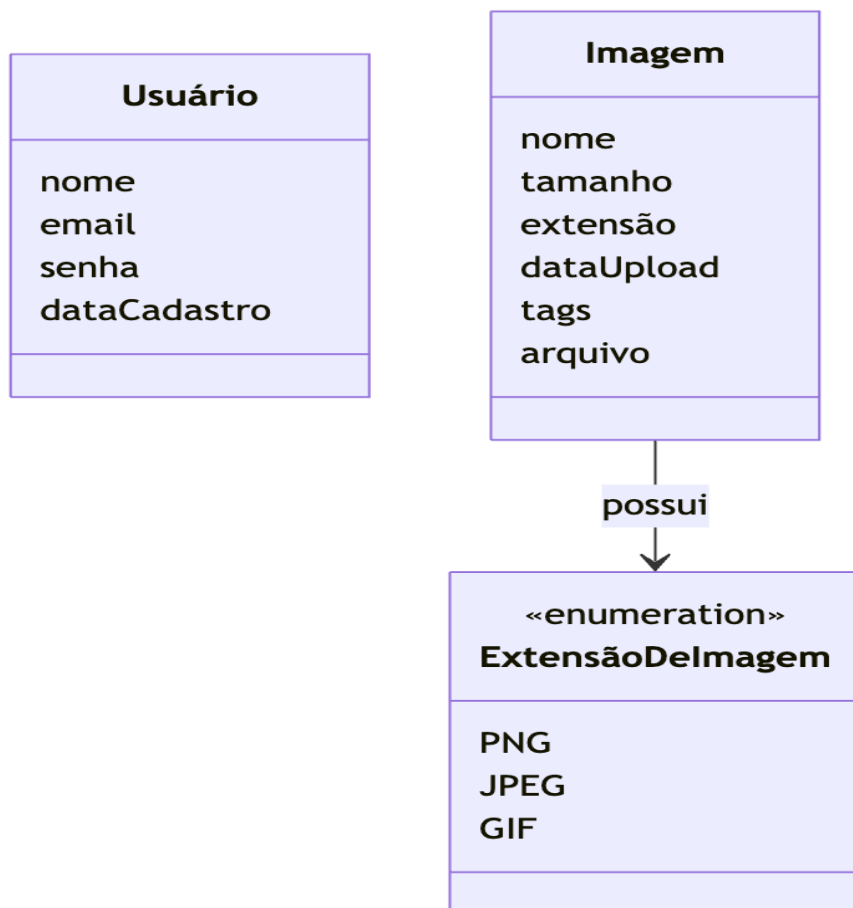
Campo	Descrição
Importância	Média
Sumário	Permite visualizar/baixar a imagem em resolução original a partir de uma miniatura.
Ator primário	Usuário
Pré-condições	Ao menos uma imagem exibida na galeria (resultado de CSU05).
Fluxo principal	1. O usuário clica sobre a miniatura de uma imagem. 2. O sistema abre a imagem original em uma nova guia do navegador.
Fluxos alternativos	Não se aplica.
Fluxos de exceção	Não se aplica.
Pós-condições	Não se aplica.
Regras de negócio	Não se aplica.
Notas de implementação	GET /v1/images/{id} — endpoint público (não exige token, ver Seção 10) — ImagesController.getImage().

Aplicações Web

Professor Lozano

Passo 7 – Modelo de Domínio (diagrama de classes de análise)

Nesta etapa (Bezerra, Cap. 5), o modelo mostra apenas os conceitos do domínio e seus atributos, sem métodos, sem tipos, sem visibilidade. É o vocabulário do negócio, extraído diretamente dos casos de uso acima.



Obs.: O modelo de domínio tal como informado não relaciona Imagem a Usuário, não existe o conceito de “autor” ou “dono” da imagem no modelo. Qualquer usuário autenticado enxerga e pesquisa o acervo inteiro.

Aplicações Web

Professor Lozano

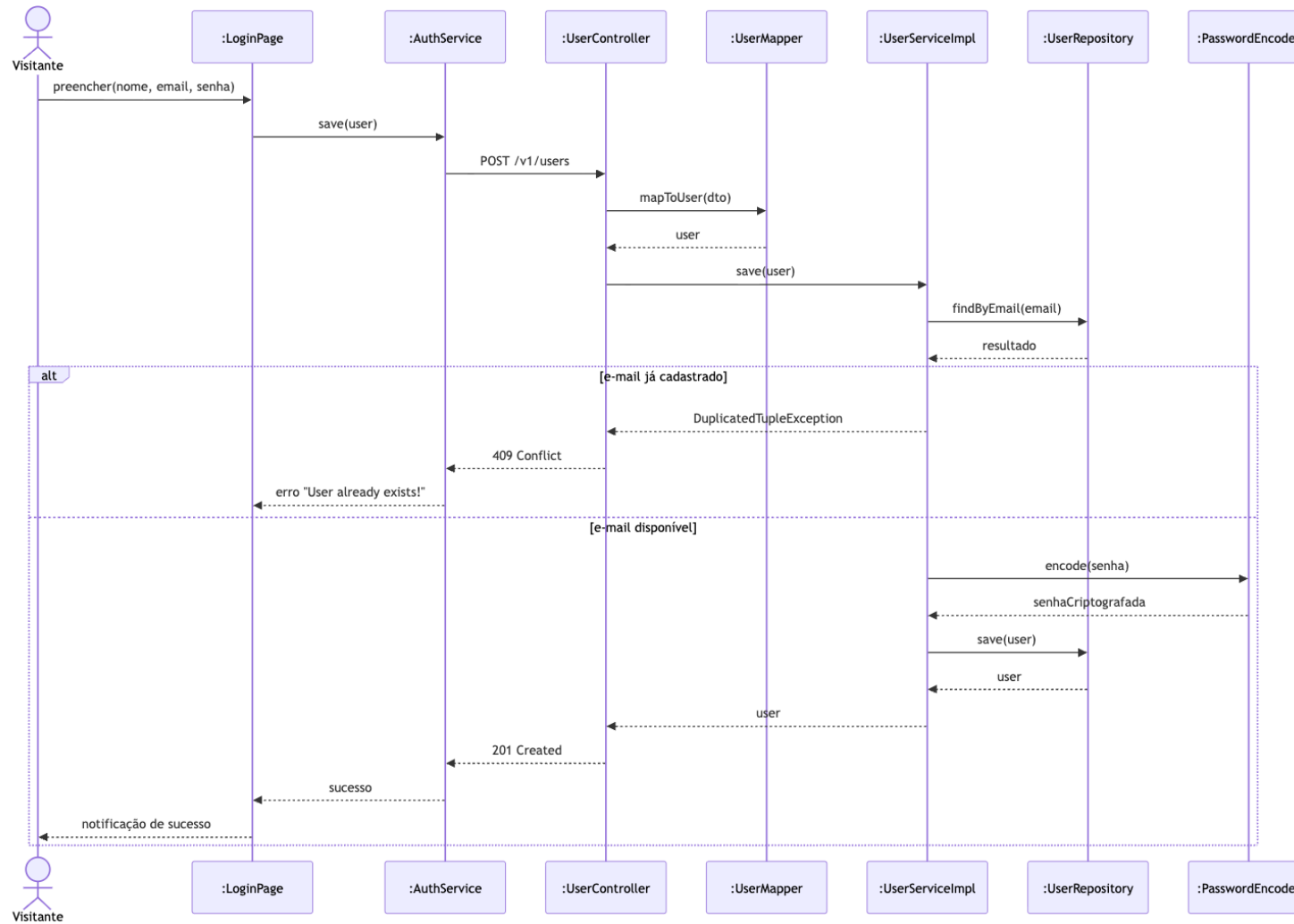
Passo 8 – Diagramas de sequência (modelagem de interações).

Para cada caso de uso, os passos do fluxo principal disparam eventos de sistema (ações do ator na fronteira do sistema). O sistema responde através de operações de sistema, alocadas a um objeto controlador, que colabora com os demais objetos do modelo. Os diagramas abaixo já mostram essa colaboração completa, com as classes reais do código, a ligação com o diagrama de classes (próximo passo) está em que cada mensagem trocada aqui vira um método de classe lá.

Aplicações Web

Professor Lozano

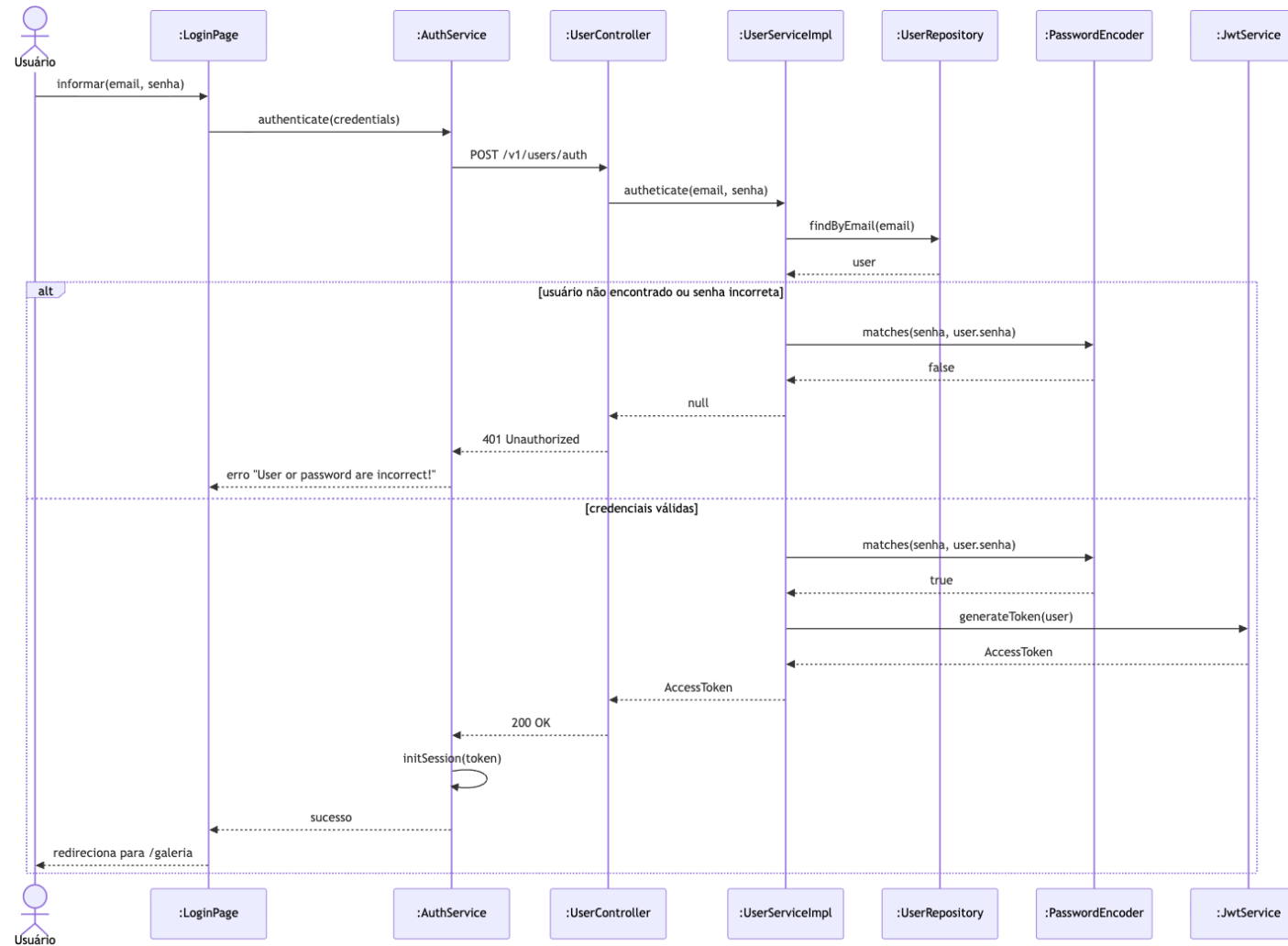
CSU01 – Cadastrar-se



Aplicações Web

Professor Lozano

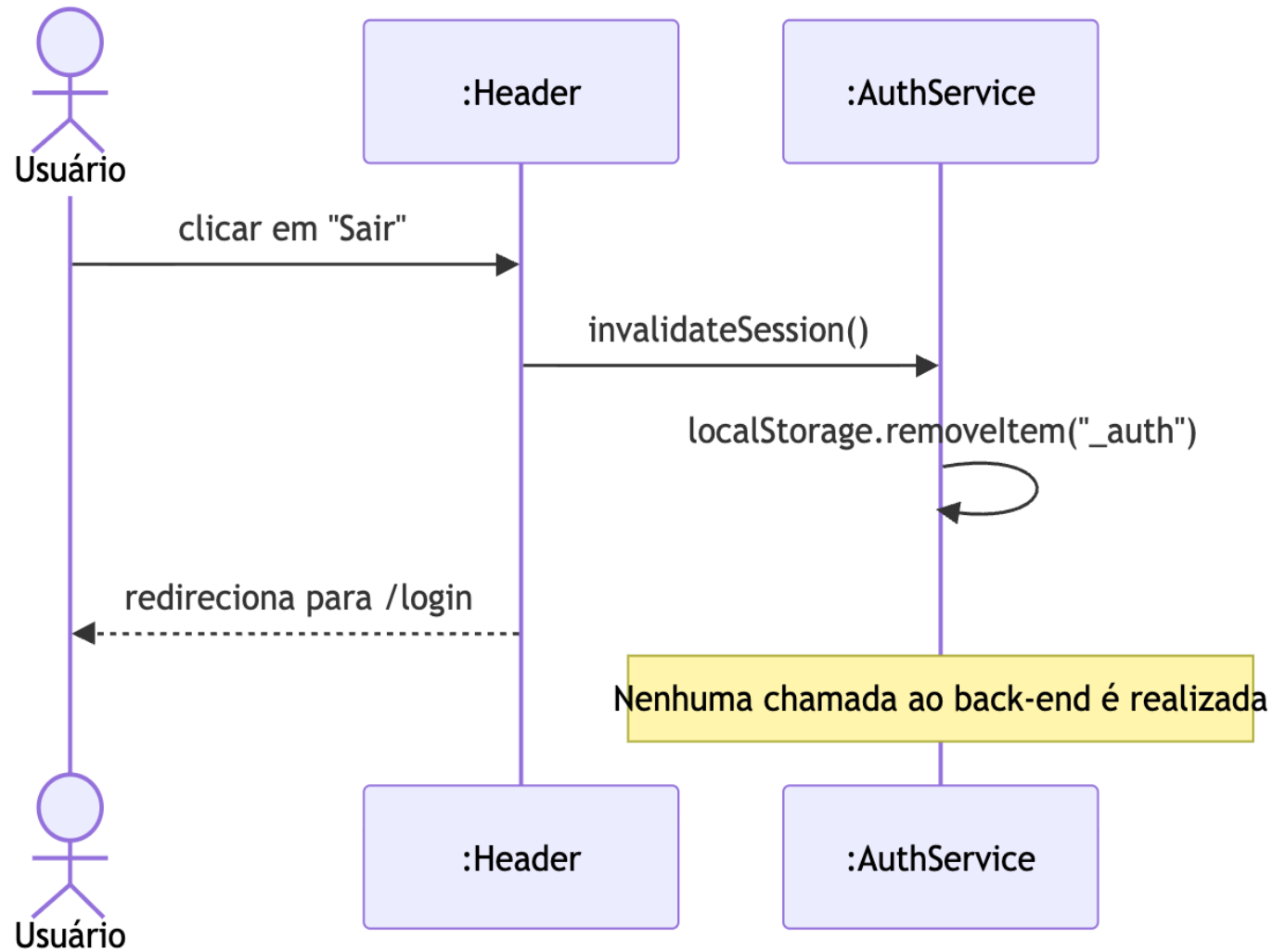
CSU02 – Autenticar-se



Aplicações Web

Professor Lozano

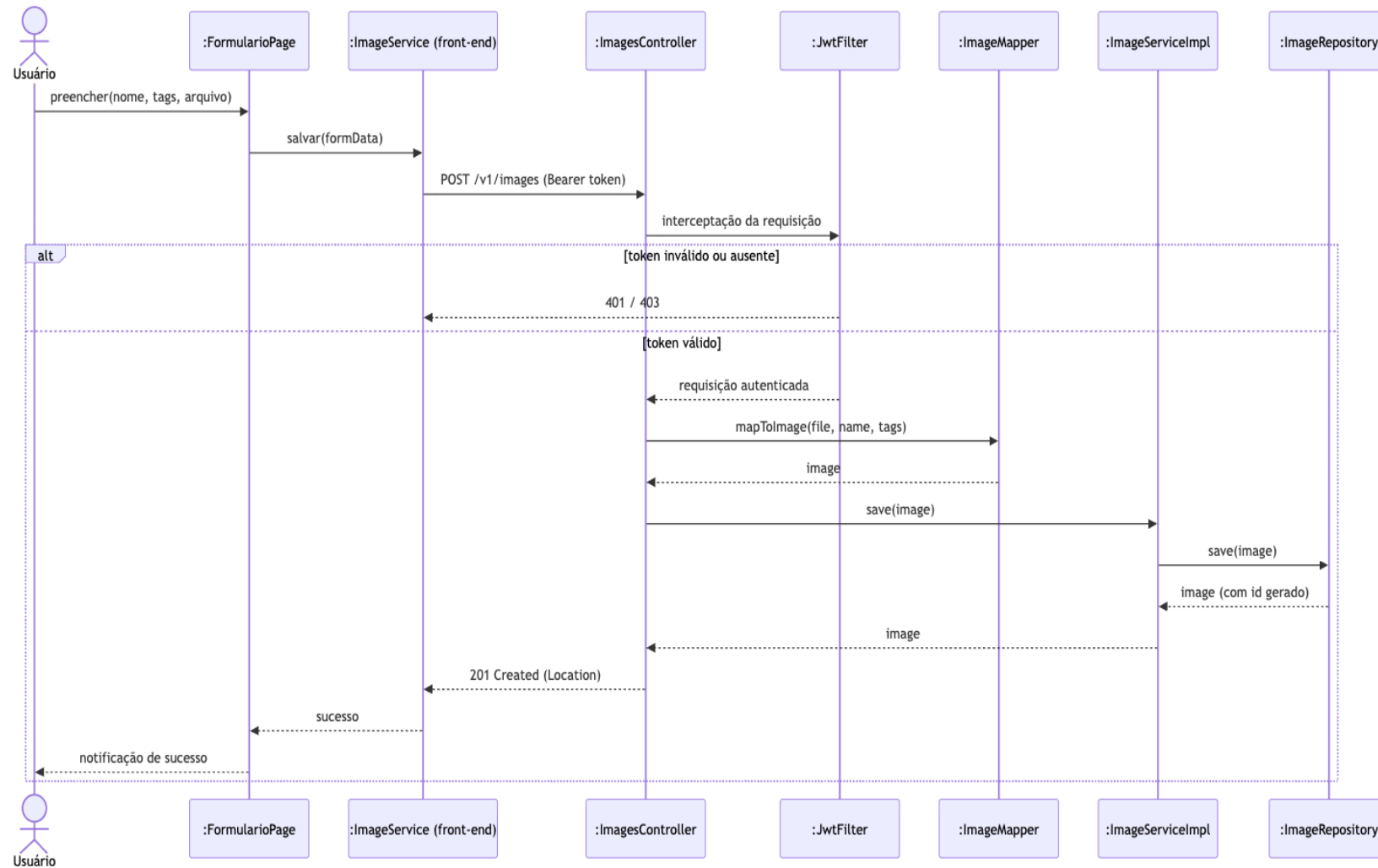
CSU03 – Encerrar Sessão



Aplicações Web

Professor Lozano

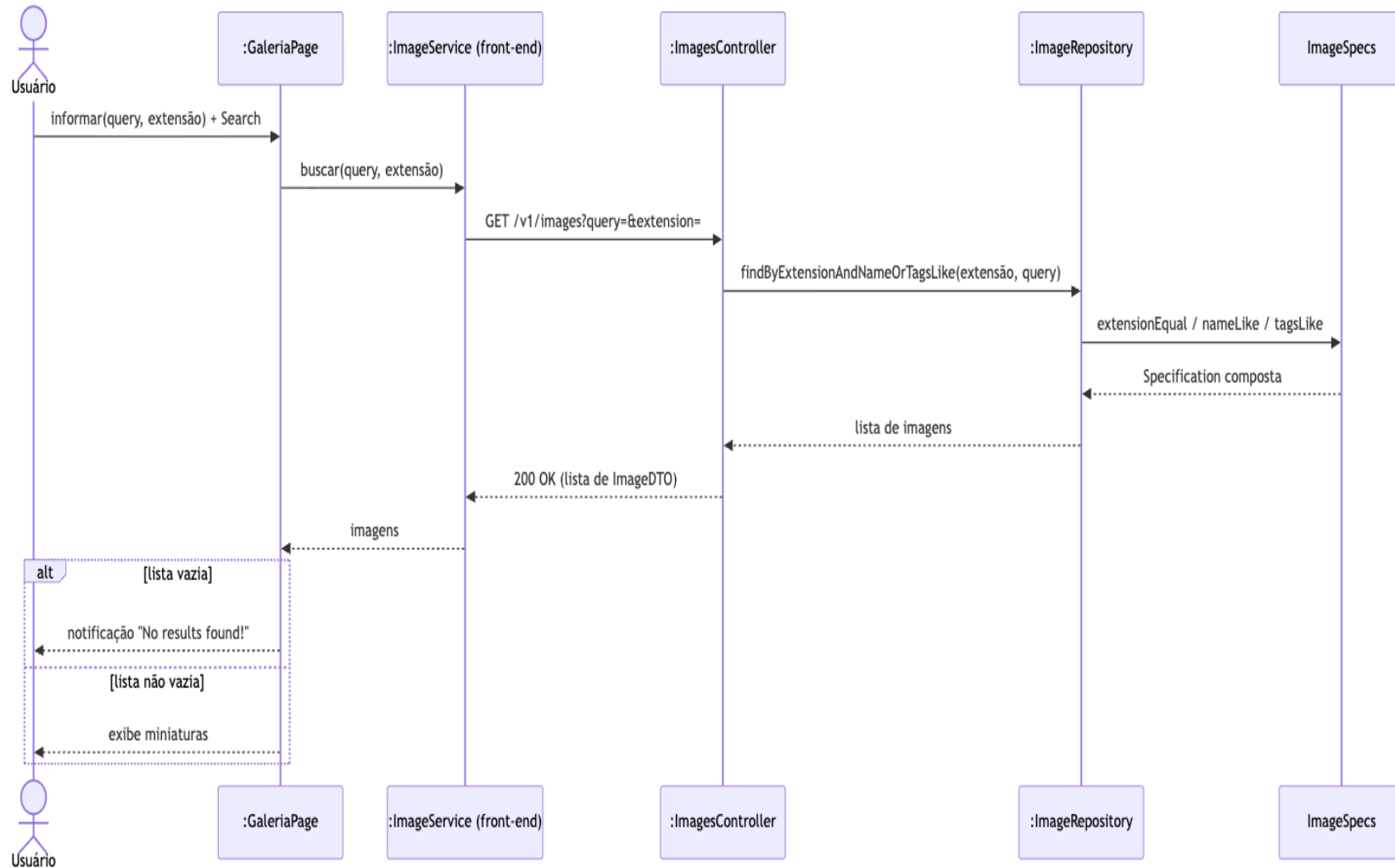
CSU04 – Publicar Imagem



Aplicações Web

Professor Lozano

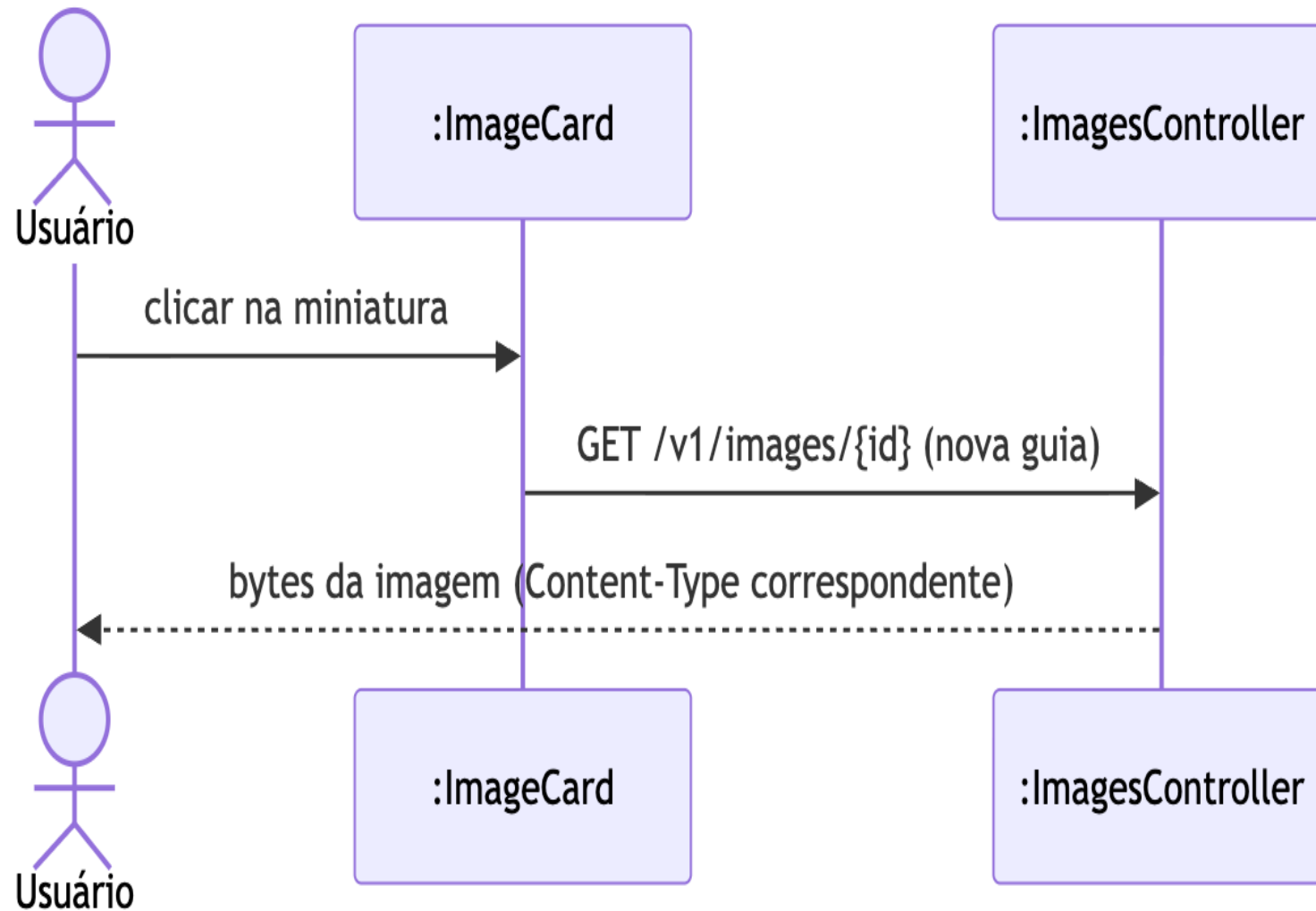
CSU05 – Pesquisar Imagens



Aplicações Web

Professor Lozano

CSU06 – Visualizar Imagem em Tamanho Real



Aplicações Web

Professor Lozano

Passo 9 – Diagramas de Classes do Projeto (modelo de implementação)

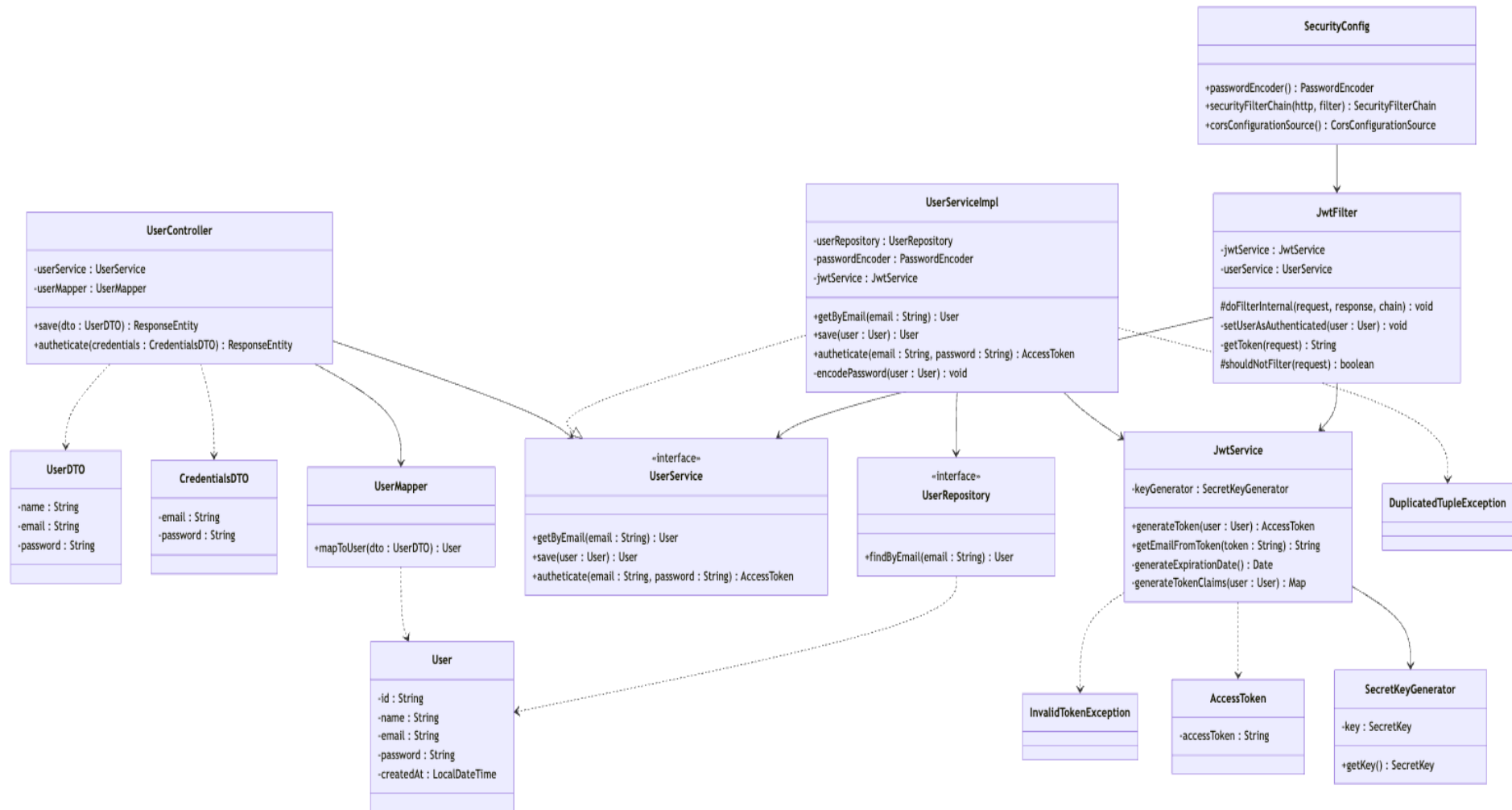
A seguir serão apresentados os diagramas de classes de projeto. Cada mensagem identificada nos diagramas de sequência aparecem aqui como uma operação da classe que a recebeu; cada objeto colaborador aparece como uma classe, com visibilidade (+ público, - privado, # protegido) e tipos reais do código-fonte.

Dividido em três diagramas por módulo, para manter a legibilidade.

Aplicações Web

Professor Lozano

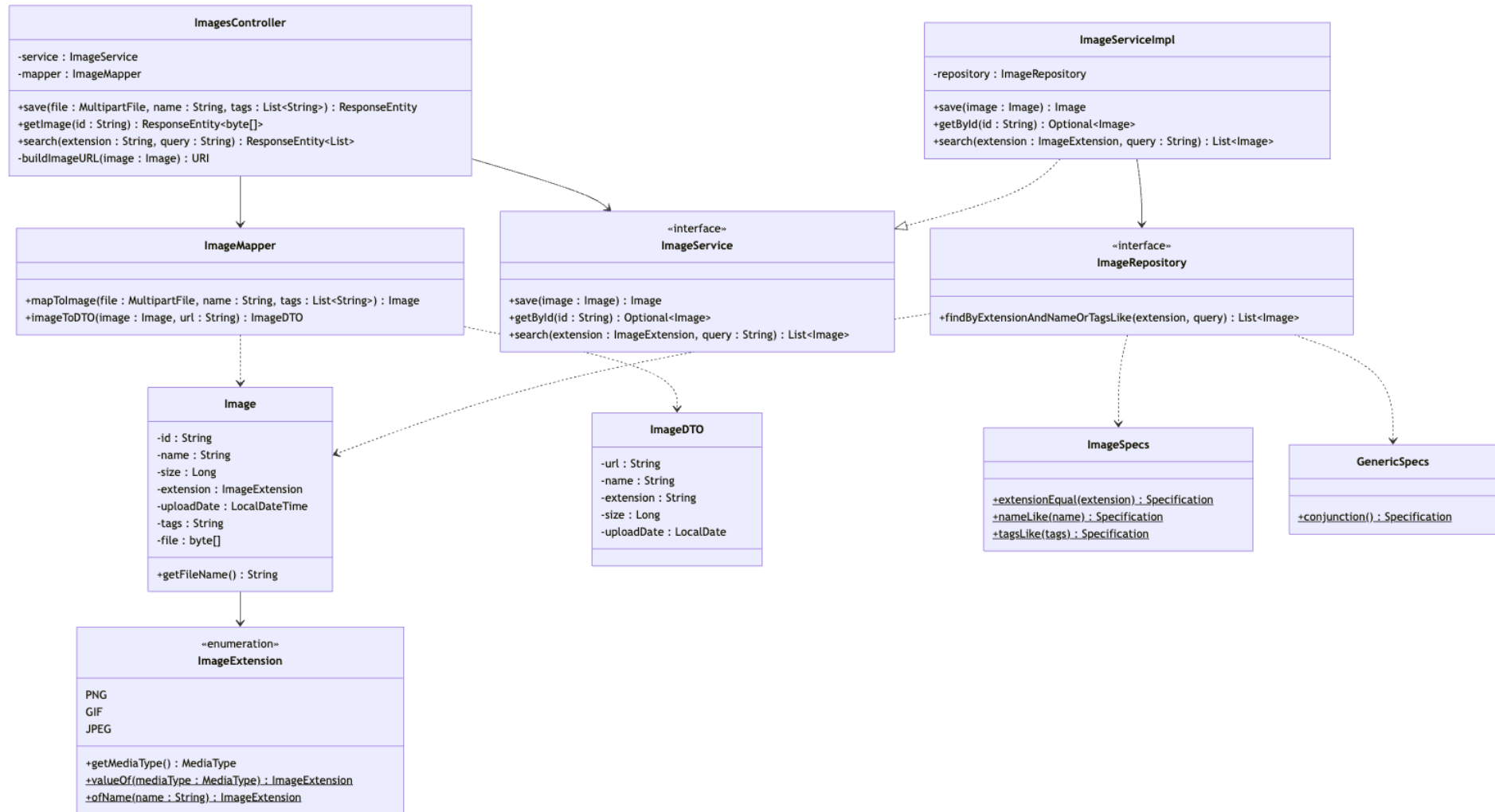
Módulos de Usuários e Segurança (Back-End).



Aplicações Web

Professor Lozano

Módulos de Imagens(Back-End).

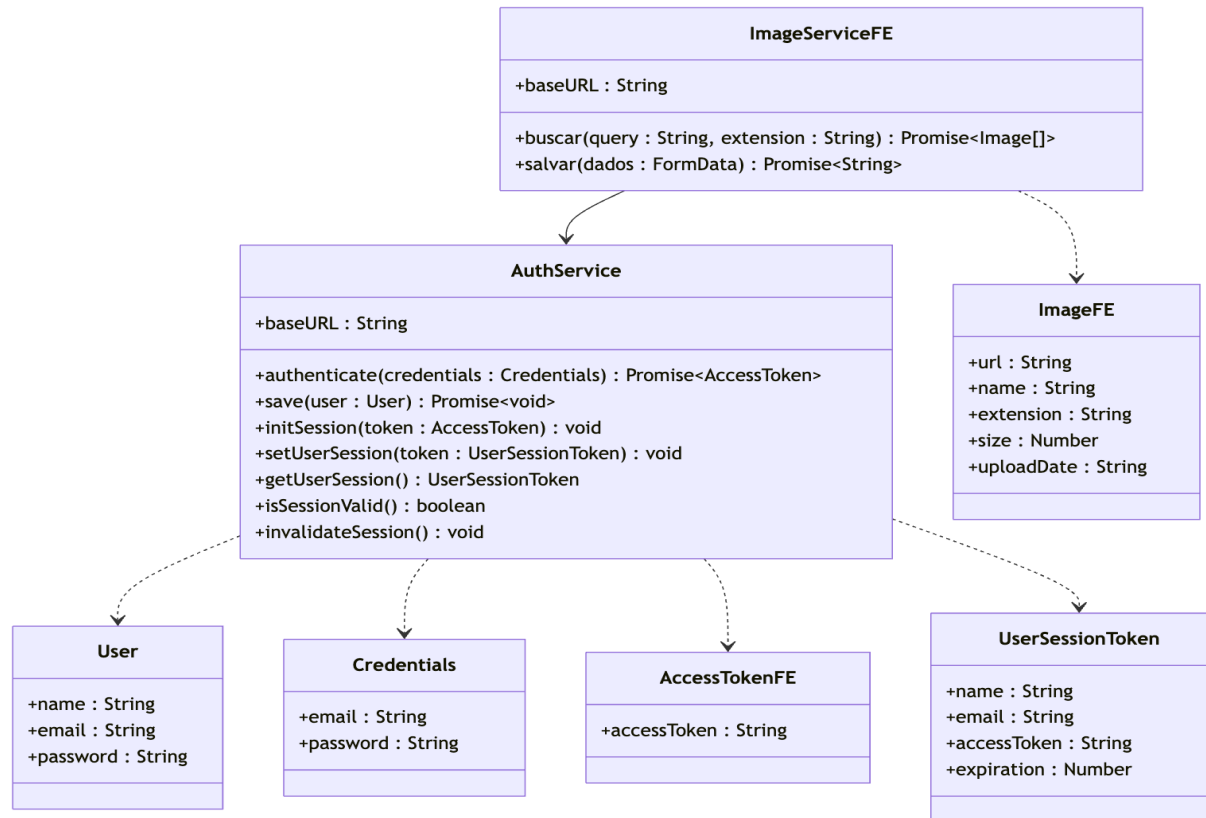


Aplicações Web

Professor Lozano

Camadas de Serviços do Front-End.

O front-end também possui classes reais (não apenas componentes React) responsáveis por falar com a API e gerenciar a sessão:



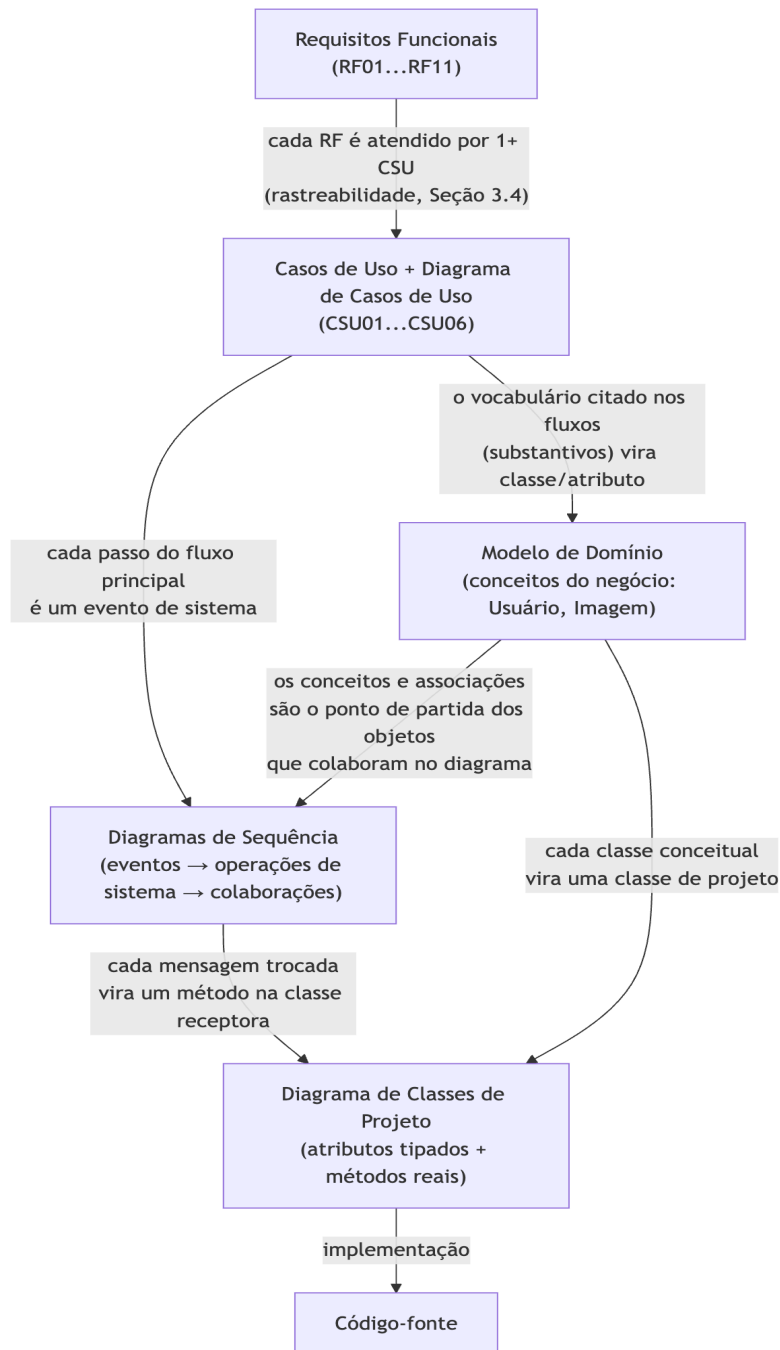
Nomeados com sufixo “FE” apenas neste diagrama para não colidir visualmente com as classes homônimas do back-end (Image, AccessToken), no código-fonte real, os nomes são Image (image.resource.ts) e AccessToken (users.resources.ts).

Aplicações Web

Professor Lozano

Passo 10 – Como os diagramas se conectam...

Esta seção responde diretamente à pergunta “o que um ajuda o outro, o que um depende do outro”:



Em outras palavras:

Aplicações Web

Professor Lozano

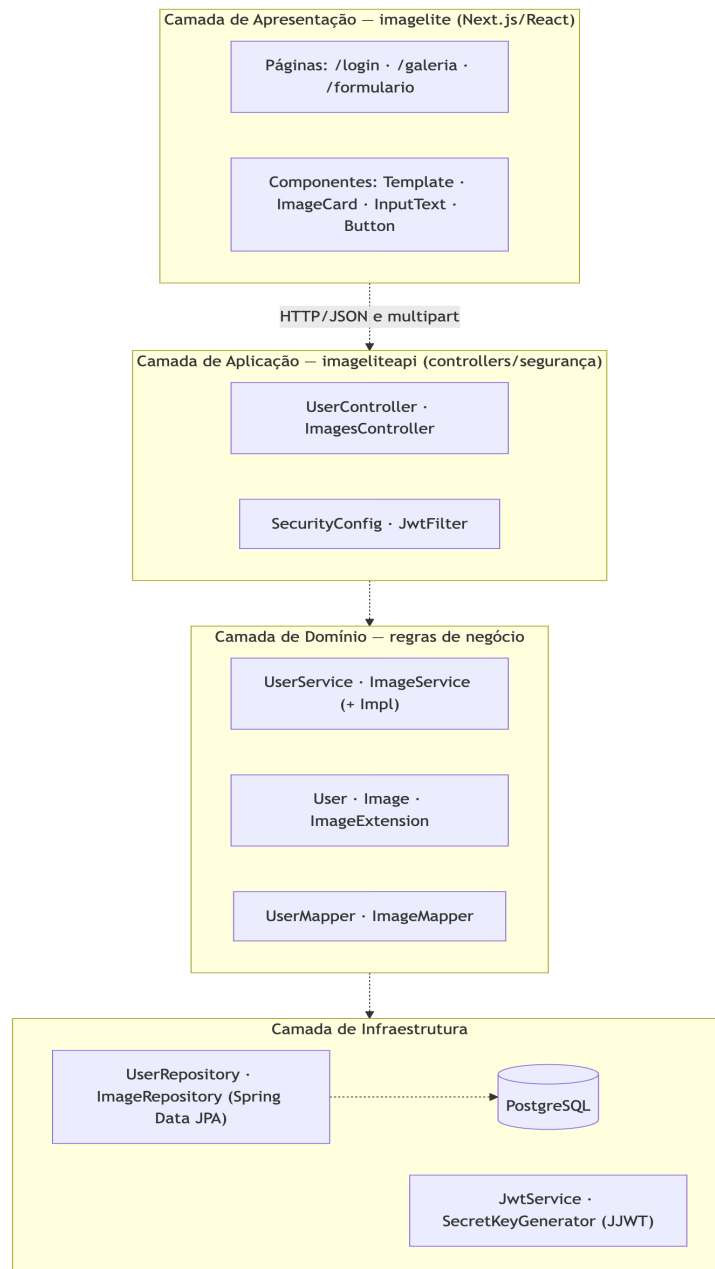
1. Casos de uso → Modelo de domínio: os casos de uso são escritos em linguagem de negócio; os substantivos que aparecem nos fluxos (“usuário”, “imagem”, “tags”, “extensão”) são candidatos a classes/atributos do modelo de domínio. É por isso que o modelo de domínio não tem métodos, ele nasce do vocabulário dos casos de uso, não do código.
2. Casos de uso → Diagramas de sequência: cada passo numerado do fluxo principal de um CSU vira, no diagrama de sequência, uma mensagem do ator para o sistema (evento de sistema). Os fluxos alternativos e de exceção do CSU viram os blocos alt/else dos diagramas, por isso CSU01 e CSU02 têm blocos alt refletindo exatamente as seções “Fluxos de exceção” das suas descrições.
3. Modelo de domínio → Diagramas de sequência: os objetos que colaboram nos diagramas de sequência são, em sua maioria, os mesmos conceitos do modelo de domínio (User, Image) mais os objetos de controle/infraestrutura que a solução técnica exige (UserController, JwtService, UserRepository...). O modelo de domínio dá o vocabulário; o diagrama de sequência descobre a dinâmica.
4. Diagramas de sequência → Diagrama de classes de projeto: toda mensagem enviada nos diagramas de sequência (findByEmail(email), generateToken(user), mapToImage(...)) reaparece nos diagramas de classe como uma operação da classe que a recebeu. É assim que o modelo de domínio “sem métodos” evolui para o diagrama de classes de projeto “com métodos”: os métodos não são inventados, eles são as mensagens descobertas na modelagem de interação.

Aplicações Web

Professor Lozano

Passo 11 – Arquitetura do Sistema

Arquitetura Lógica em Camadas



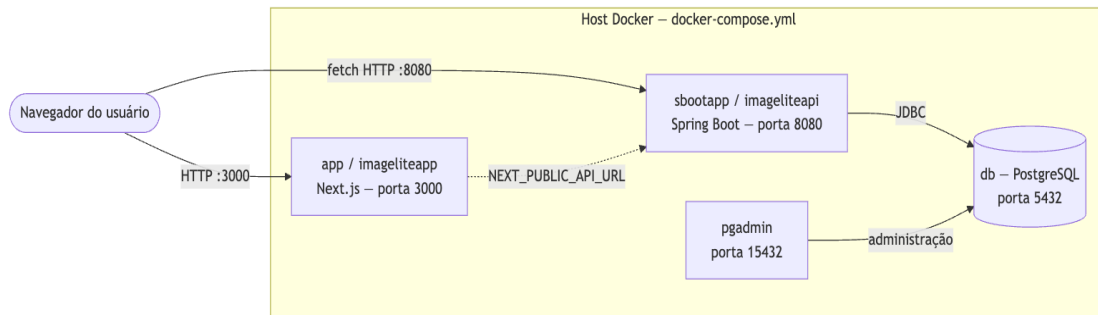
Mapeamento para 4 camadas:

- Apresentação = front-end Next.js;
- Aplicação = controllers REST + filtro de segurança;
- Domínio = serviços + entidades + regras de negócio;
- Infraestrutura = repositórios JPA, geração/validação de JWT e o banco PostgreSQL.

Aplicações Web

Professor Lozano

Arquitetura Lógica em Camadas



Passo 12 – Modelo Entidade-Relacionamento (DER)

auth_user		
varchar	id	PK
varchar	name	
varchar	email	
varchar	password	
timestamp	created_at	

image		
varchar	id	PK
varchar	name	
bigint	size	
varchar	extension	
timestamp	upload_date	
varchar	tags	
oid	file	

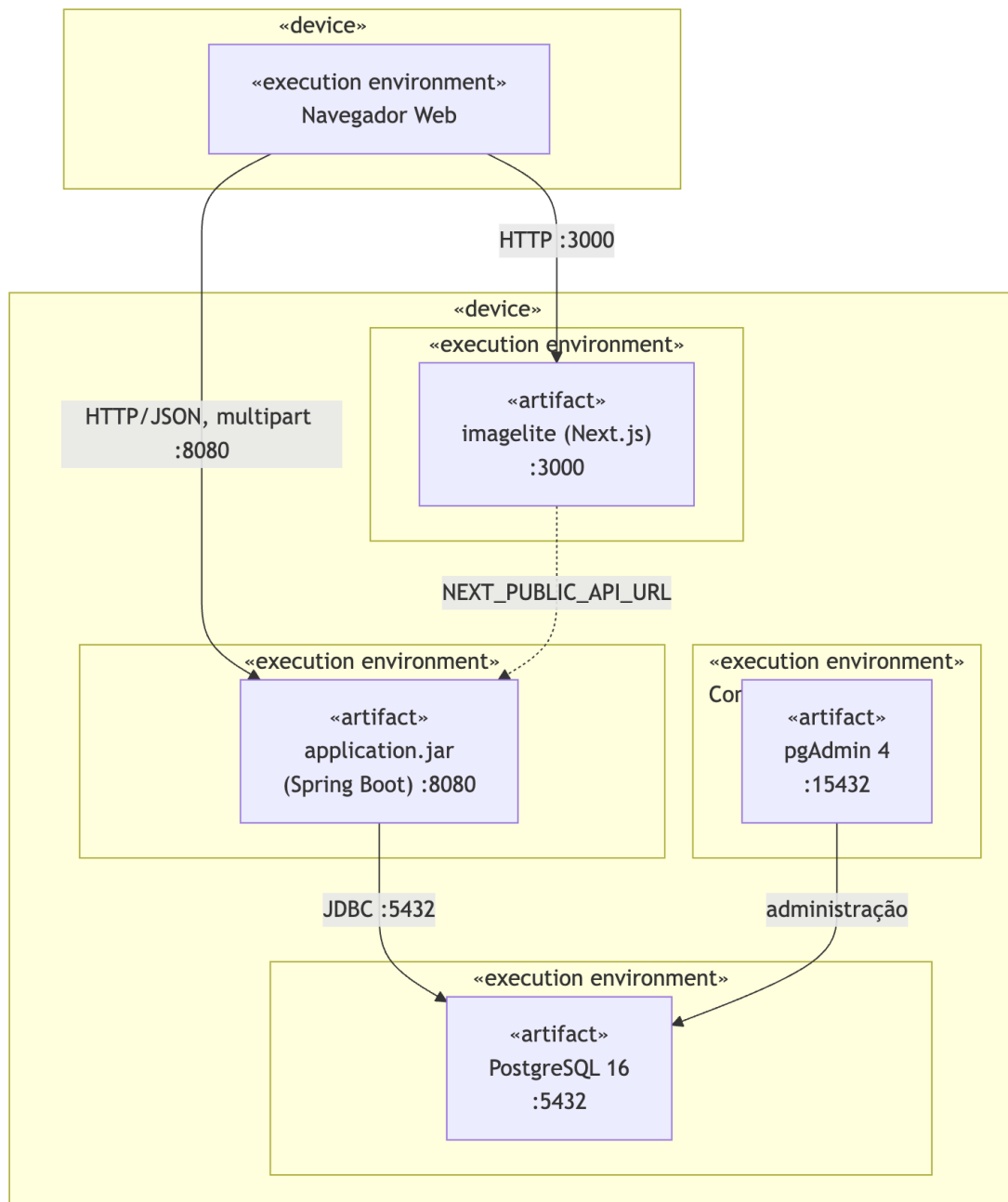
Não há chave estrangeira entre `image` e `auth_user`, pois nenhuma relação de posse entre usuário e imagem no modelo atual. `file` é armazenado como `OID` (Large Object do PostgreSQL), não como `byte`, a decisão do Hibernate ao mapear `byte` anotado com `@Lob` neste projeto.

Aplicações Web

Professor Lozano

Passo 13 – Diagrama de Implantação

Versão em notação UML formal (nós, artefatos e caminhos de comunicação) do mesmo cenário físico já apresentado de forma simplificada anteriormente. O Mermaid não possui um tipo de diagrama de implantação nativo; os estereótipos UML foram preservados como texto («device», «execution environment», «artifact»).



Aplicações Web

Professor Lozano

Passo 15 – Diagrama de atividades — Publicar Imagem (CSU04)

Representa em fluxo de decisão o mesmo caso de uso já descrito anteriormente e modelado como sequência, útil para visualizar os pontos de validação (RN03, RN04, RN05) como decisões, complementando as duas outras visões do mesmo caso de uso.

Aplicações Web

Professor Lozano

